

Holdfast AR: Technical Documentation

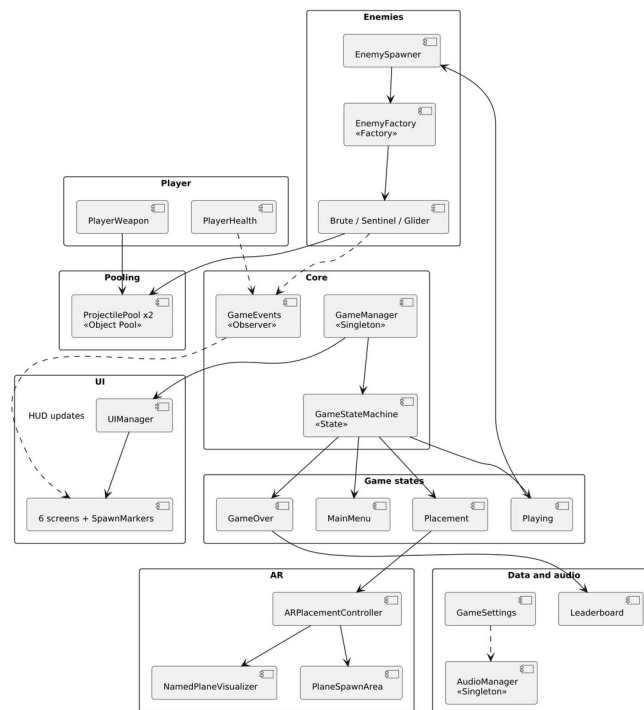
Joshua Chukwuebuka Moses. Repository: github.com/JOSHUALU/holdfast-ar

Holdfast AR is a first person survival shooter for phones. You play the last colonist on a planet called Kepler-9. You scan your real floor, drop your landing dome on it, and hold off three kinds of alien attackers with a ray gun until the evac ship arrives. If the timer runs out, you win. If your health hits zero, the dome falls.

1. Architecture overview

Everything runs in one scene. A GameManager owns a small state machine that moves between four states: Main Menu, Placement, Playing and Game Over. Each state sets itself up when it starts and cleans up when it ends, so leaving Playing always wipes every enemy and bullet. Gameplay code never calls the UI. It raises events on a static GameEvents class and the HUD listens, which meant I could rebuild the whole UI later without touching any combat code.

The UI is built from code by UIManager inside a 1080 by 1920 portrait frame that fits the phone's safe area, so every screen shares the same theme and layout on any phone size.



2. OOP structure

Encapsulation. Data sits in private fields behind read only properties. The score only changes through `GameSession.RegisterKill()`, health only through `PlayerHealth.TakeDamage()`, and Leaderboard and GameSettings hide how they save to `PlayerPrefs`. Inspector values use `[SerializeField] private`.

Abstraction. `IDamageable` lets bullets and attacks hurt anything without knowing what it is. `IPoolable` gives pooled objects their reset hooks. `Enemy`, `GameState` and `UIPanel` are abstract base classes that set the rules without the details.

Inheritance. `MeleeEnemy`, `ShooterEnemy` and `DiverEnemy` all extend `Enemy`, which already handles health, chasing, turning, cooldowns, hit feedback and death. The four states extend `GameState`, the six screens extend `UIPanel`, and `GameManager` and `AudioManager` extend `Singleton<T>`.

Polymorphism. `Enemy.Update()` calls the abstract `StoppingDistance` and `Attack()` without knowing which enemy it is running. The Brute walks up and punches, the Sentinel stops and

shoots, and the Glider overrides `Animate()` to fly and dive at the phone. The state machine only ever calls `Enter`, `Tick` and `Exit`, and `UIManager` only calls `Show` and `Hide`.

The three enemies look and fight differently. The **Brute** (melee) is a green alien that walks up to 0.35 m away and punches for 15 damage when you're within 0.5 m, every 1.1 seconds. It takes 3 shots and gives 100 points. The **Sentinel** (shooter) is a yellow bug mech that stops at 1.4 m and fires projectiles from up to 2.2 m, doing 8 damage per shot every 1.6 seconds. It takes 5 shots and gives 150 points. The **Glider** (diver) is a purple flyer that never stops: it dives at the phone and bursts on impact for 20 damage. One shot kills it and it gives 75 points.

3. Design patterns used and why

Object Pool (`ObjectPool<T>`, `ProjectilePool`, `Projectile`). The gun fires six shots a second. Creating and destroying bullets that often causes garbage collection spikes, and on a phone that shows up as stutter, which also hurts AR tracking.

State (`GameStateMachine`, `GameState` and four states). Each phase owns its own setup and cleanup, so there is no giant switch statement and the `Start`, `Play`, `End` flow is easy to follow.

Singleton (`GameManager`, `AudioManager`). There should only ever be one game coordinator and one sound owner, and both need to be easy to reach. Duplicates destroy themselves.

Factory (`EnemyFactory`). One place turns an enemy type into the right prefab and hands it what it needs. Adding the Glider only meant adding a new prefab and type.

Observer (`GameEvents`, `HudPanel`, `SpawnMarkers`, `GameSettings`). Health, score, kills, time and spawns are broadcast as events. The HUD subscribes when it opens and unsubscribes when it closes, and settings changes tell `AudioManager` to update the music straight away.

4. Object Pool implementation

When the game loads, `ProjectilePool` creates 40 player bullets and 30 enemy bullets, switches them off and keeps them in a stack. That is the only time a bullet is ever instantiated. Firing takes one off the stack, places it, switches it on and calls `OnTakenFromPool()`. If the stack is ever empty, the oldest bullet in flight is recycled instead of creating a new one.

When a bullet hits something or reaches its 3 second lifetime, it hands itself back. The pool calls `OnReturnedToPool()`, which clears its age, speed, damage, direction, team and trail, then switches it off and puts it back on the stack. Double returns are ignored, and at the end of a match `ReleaseAll()` pulls every bullet back. Each frame a bullet sphere-casts along its path, so fast bullets can't skip through small targets, and it only damages things on the other team.

5. Sound system structure and sources

`AudioManager` is the only object in the game with `AudioSources`, so enemies and bullets don't each carry their own. It has one 2D source for player and UI sounds (using `PlayOneShot`, so rapid shots overlap cleanly), one looping music source, and a fixed set of six 3D sources that move to wherever an enemy sound happens. Clips load by name from `Resources/Audio`, each with its own volume and a slight random pitch so repeats don't sound robotic. The Settings screen lets the player set sound effects and music to off, low or high.

The player shoot sound plays in `PlayerWeapon.Fire()`, and the player death sound in `PlayerHealth.TakeDamage()` when health reaches zero. The enemy spawn sound plays in `EnemySpawner.SpawnOne()`, the Sentinel's shot in `ShooterEnemy.Attack()`, and the melee hit in `MeleeEnemy.Attack()`. The enemy sounds are 3D, so you can hear where they are. On top of those there are sounds for getting hurt, enemy hits and deaths, button clicks, placing the dome, winning, and an ambient music loop.

I made all 12 sounds myself with a small Python script (`Tools/make_audio.py`) that builds them from sine, square and saw waves, pitch sweeps and filtered noise, saved as 16-bit WAV files. The

ambient loop only uses frequencies that finish whole cycles in 16 seconds, so it loops without a click.

6. AR setup

The plane manager only looks for horizontal planes. I replaced Unity's default plane look with my own NamedPlaneVisualizer, which builds a mesh from each plane's outline and tiles a texture across it with JOSHUA CHUKWUEBUKA MOSES written on it. It only shows while a plane is being tracked.

A tap raycasts against the planes and attaches an anchor where it hits, and the dome is parented to that anchor. After that, extra taps do nothing, plane detection stops and the trackers hide. When the dome lands, I save every plane's outline, so enemies only spawn at points inside a real plane, 0.9 to 2.2 m from the dome and at least 1.1 m from the player. A small marker on the HUD shows where each one just appeared. The leaderboard saves each match as JSON in PlayerPrefs, newest first, and keeps the latest 5.

7. Performance metrics

I measured this in the Unity 6 editor with XR Simulation at 1080 by 1920, with six enemies on screen and the fire button held down for 15 seconds straight. It averaged 57 FPS (the editor caps close to 60), with each frame taking about 17 ms and rendering only about 2.3 ms of that. The scene used 56 draw calls, 14 SetPass calls and about 30,000 triangles, which is light because the models are low poly. Most importantly, there were zero garbage collections during the whole 15 seconds of shooting, and the scene had 70 bullet objects before and after, so nothing was created or destroyed during play. The Android APK is 45 MB.

Appendix

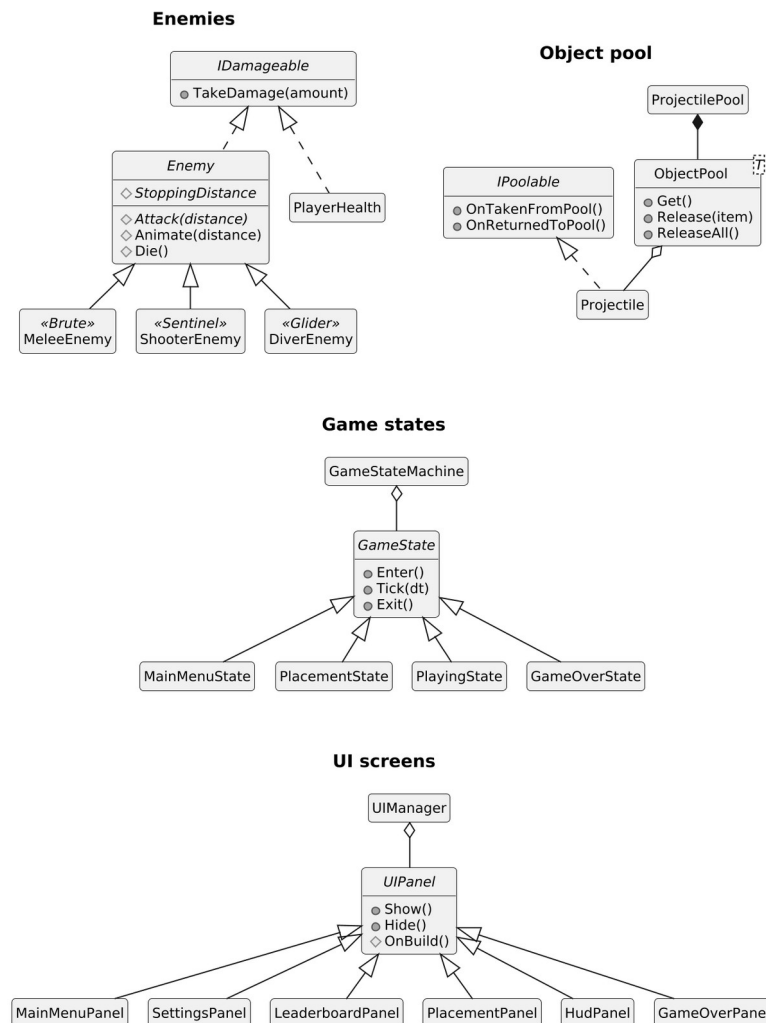
Difficulty levels (bonus)

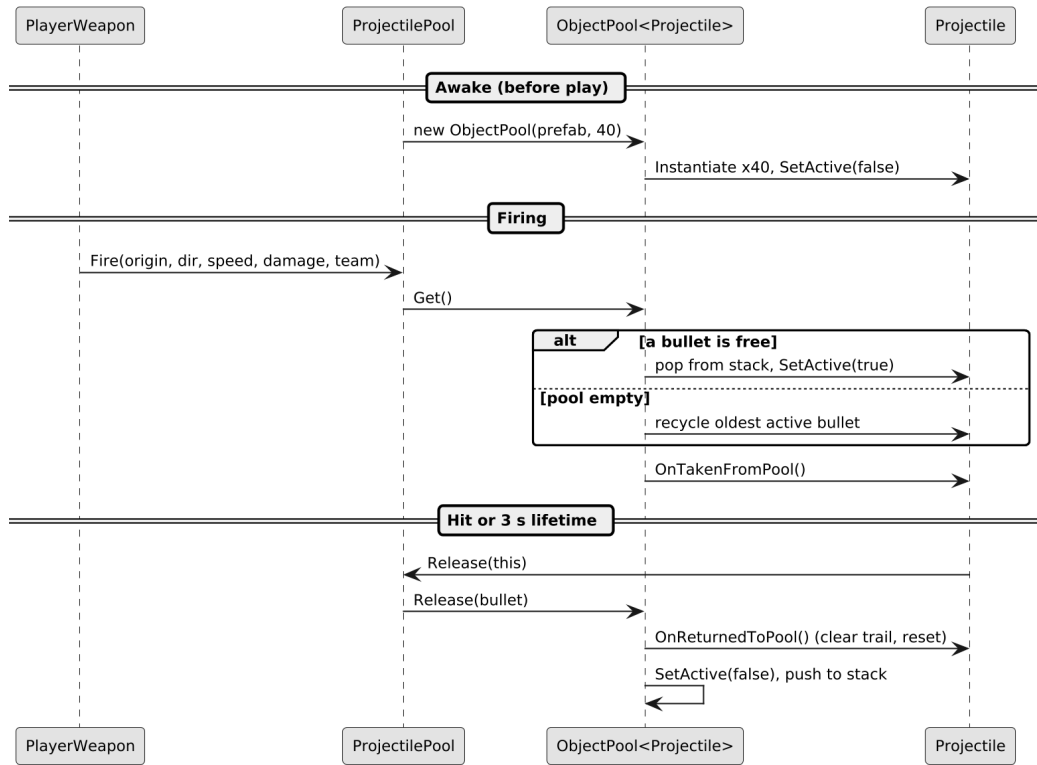
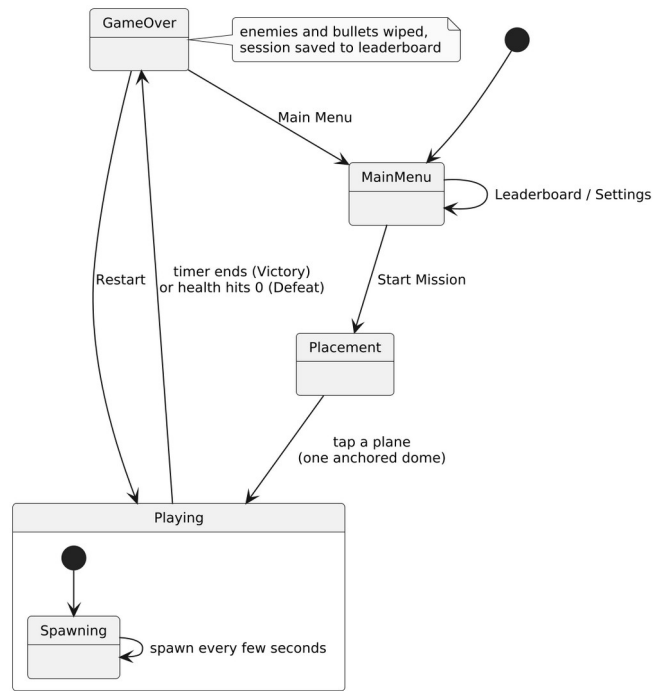
Easy gives a 60 second match, 150 health, a spawn every 3.2 seconds, up to 4 enemies at once, slower and weaker enemies, and a 0.8 score multiplier. Normal is 90 seconds, 100 health, a spawn every 2.4 seconds and up to 6 enemies. Hard is 120 seconds, 80 health, a spawn every 1.6 seconds, up to 9 enemies that are 30% faster and hit 40% harder, and a 1.5 score multiplier. Harder levels also send more Sentinels and Gliders. Each level is a DifficultySettings ScriptableObject, so balancing never needs a code change.

Assets and sources

The Brute, Sentinel, Glider, dome, rocks, plants and solar panel come from the Ultimate Space Kit by Quaternius, and the ray gun from the Sci-Fi Gun Pack by Quaternius, both from poly.pizza under CC0. The panels, buttons, bars, crosshair and spawn markers come from the UI Pack (Space Expansion) by Kenney, and the font is Kenney Future, both from kenney.nl under CC0. I made the sounds, the plane tracker texture, the app icon and the animators myself.

Diagrams





Screenshots

